

# Agentproof: Static Graph Verification for Agent Workflow Graphs

|                   |                   |
|-------------------|-------------------|
| First Last        | Second Author     |
| Affiliation       | Affiliation       |
| email@example.com | email@example.com |

## Abstract

Modern agent frameworks encode tool-using behavior as explicit workflow graphs: nodes represent LLM calls, tools, routers, or human approval steps; edges represent possible control-flow transitions. Most “agent safety” products enforce policies at runtime by intercepting calls, adding latency and detecting violations only after execution is underway. We present *Agentproof*, a static verification approach that treats an agent workflow as a formal directed graph and checks safety properties *before deployment*. Agentproof provides (i) a unified abstract graph model across popular frameworks, (ii) extractors for several agent orchestration libraries, (iii) structural analyses for common workflow defects (unreachable nodes, dead ends, missing human gates), and (iv) a small temporal policy DSL compiled into deterministic finite automata for fast event-trace evaluation. We demonstrate the approach on representative workflow graphs built using real framework APIs and report microbenchmarks for extraction and monitor compilation/evaluation.

## 1 Introduction

Agent systems increasingly operate over external tools: retrieving data, writing tickets, sending messages, mutating databases, and triggering workflows. In most orchestration frameworks, this behavior is represented explicitly as a workflow graph (or a closely related state machine): nodes encode computation steps and edges encode allowable transitions.

Despite this structural representation, safety enforcement is commonly applied as a runtime gate—a wrapper around tool calls that checks a policy and either allows or blocks execution. Runtime checks are valuable, but they impose latency on every step and often detect violations only after the agent has already reached unsafe parts of its workflow.

This paper argues for a complementary perspective: *pre-deployment static verification* of agent workflow graphs. Static verification is successful in traditional systems because it reasons over an explicit model of behavior [5]. When agent behavior is graph-structured, many safety properties reduce to reachability, isolation, and temporal constraints over the graph topology.

We present *Agentproof*, a practical toolkit for static analysis of agent workflows. Agentproof focuses on properties that are (a) expressible over the workflow graph and (b) independent of LLM text generation semantics. In particular, it verifies the *structure* of possible executions, not the content of model outputs.

**Contributions.** Agentproof makes the following contributions:

- A unified abstract workflow graph model (**AgentGraph**) with typed nodes and edges suitable for cross-framework analysis.

- Extractors from several popular agent frameworks into this model, enabling tool- and control-flow aware inspection before deployment.
- A set of structural checks for common workflow defects (unreachable nodes, dead ends, missing human approval gates, and tool declaration issues).
- A small temporal policy DSL compiled to deterministic finite automata for fast event-trace evaluation inspired by temporal logic monitoring [6].

**Scope.** Agentproof is designed to validate workflow design decisions early: “can this graph reach a destructive tool?” or “must human review occur between two sensitive actions?” It does not attempt to prove semantic properties of LLM outputs (e.g., truthfulness), which remain inherently non-deterministic.

## 2 Background

### 2.1 Agent workflow graphs

Many agent frameworks represent behavior as a directed graph, sometimes called a *state graph* or *workflow graph*. Nodes correspond to steps such as: an LLM call, a tool invocation, a router/selector that branches based on state, or a human approval node. Edges correspond to transitions between these steps, including conditional branches, parallel fan-outs, or loop back-edges.

This explicit control-flow model is a key difference between graph-orchestrated agents and monolithic “single prompt” LLM applications: verification can be performed over the topology without running the agent.

### 2.2 Structural verification

Structural properties are those that depend only on graph topology and type annotations. Typical examples include: (i) reachability (can a node be reached from the entry point?), (ii) dead ends (nodes with no outgoing transitions), (iii) isolation (must a sensitive tool be preceded by a gate node?), and (iv) sanity constraints on router nodes.

These checks are inexpensive: they can be implemented with standard graph algorithms (BFS/DFS) and run in time linear in the number of nodes and edges.

### 2.3 Temporal monitors

Temporal logics such as linear temporal logic (LTL) provide a formal language to specify constraints over execution traces [6]. A common verification route is to compile temporal constraints into automata and then reason over traces or over the product of a system model and the automaton.

Agentproof implements a lightweight monitoring path: a small temporal policy DSL is compiled into a deterministic finite automaton (DFA) and evaluated over an event stream representing tool calls and other agent actions. This design is inspired by automata-theoretic monitoring approaches (e.g., [7]) but intentionally focuses on a small, practical fragment.

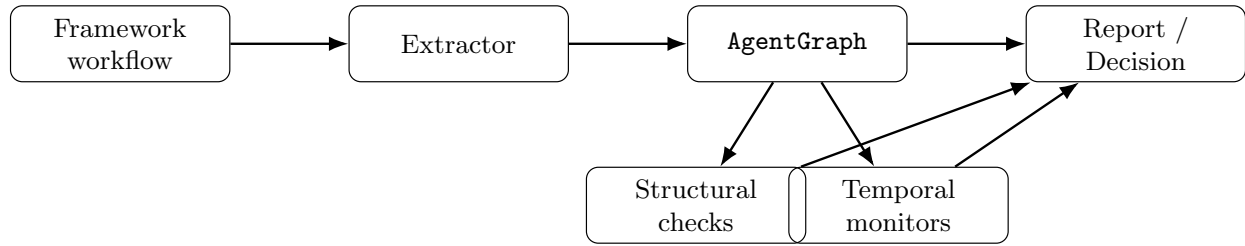


Figure 1: Agentproof pipeline: extract a framework workflow into an abstract graph model, run structural verification, and compile temporal policies into monitors for trace evaluation.

## 2.4 Related work (brief)

Runtime guardrails, policy engines, and tool-call interceptors enforce safety online, often with logging and audit support. Agentproof is complementary: it targets *static* properties of the workflow design that can be checked before deployment, reducing the chance that unsafe topologies ship in the first place.

## 3 System overview

Figure 1 summarizes Agentproof’s workflow. The system ingests a framework-specific agent definition, extracts an abstract graph model, and then runs a suite of structural and temporal checks.

**Step 1: Obtain a workflow object.** Agentproof targets agent orchestration libraries that expose control flow as a graph or composition tree. In this repository we include case studies built using real APIs from LangGraph, Google ADK, AutoGen, and CrewAI [4, 3, 1, 2].

**Step 2: Extract an AgentGraph.** Each framework has an extractor that produces a common representation: a set of typed nodes and typed edges with optional metadata (e.g., tool names bound to a node). This makes downstream analyses framework-agnostic.

**Step 3: Run structural checks.** Structural checks detect workflow defects that do not require executing the agent: unreachable nodes, dead ends, routing shape mismatches, and the presence of human-in-the-loop nodes when required by policy.

**Step 4: Compile temporal policies.** Agentproof supports a small temporal policy DSL which is compiled to a DFA. Policies are written against abstract events (tool calls, action tags, or decisions) rather than framework internals.

**Step 5: Evaluate monitors over traces.** Monitors can be evaluated on offline simulated traces (as in our examples) or integrated into a runtime that emits events. Agentproof maps monitor violations to handling levels (warn, block, halt, escalate) to support different operational postures.

**Non-goals.** Agentproof does not attempt to prove properties about the natural language outputs of LLM calls. Its verification surface is intentionally the workflow graph and event stream, not the semantic correctness of generated text.

## 4 Graph model and extraction

Agentproof defines a framework-agnostic workflow representation, **AgentGraph**, with typed nodes and edges. This abstraction is the common surface for verification.

### 4.1 Abstract model

An **AgentGraph** consists of a set of nodes  $V$ , edges  $E$ , an entry node identifier, and one or more exit identifiers. Each node has a *kind*: ENTRY, EXIT, TOOL, LLM, ROUTER, HUMAN, SUBGRAPH, or PASSTHROUGH. Each edge has a *kind*: DIRECT, CONDITIONAL, PARALLEL, or LOOP.

Tool nodes may include a list of bound tool names (e.g., ("http.get", "s3.download")). Edges may include optional condition labels for conditional branches.

### 4.2 Framework extractors

Agentproof includes extractors that map framework objects into this model:

**LangGraph.** LangGraph exposes a compiled state graph with sentinel start/end nodes. The extractor identifies entry/exit sentinels, infers node kind from tool bindings and naming heuristics (e.g., human review nodes), and encodes conditional edges when the underlying framework marks an edge as conditional.

**Google ADK.** Google ADK composes agents as a tree of sequential, parallel, and loop agents. The extractor performs a tree walk, emitting subgraph nodes for composite agents and encoding sequential chains, parallel fan-outs, and loop back-edges.

**AutoGen.** AutoGen team topologies can be provided as a team object (e.g., round-robin) or as an explicit list of agents plus a transition relation. The extractor constructs a directed graph of speaker transitions, synthesizes entry/exit, and connects leaf speakers to exit.

**CrewAI.** CrewAI describes task pipelines (sequential or hierarchical). The extractor creates a node per task, uses process mode to add sequential edges or manager routing edges, and incorporates context dependencies between tasks.

### 4.3 Example workflow graph

Figure 2 shows a small extracted workflow used throughout the paper to illustrate typed nodes and conditional branches.

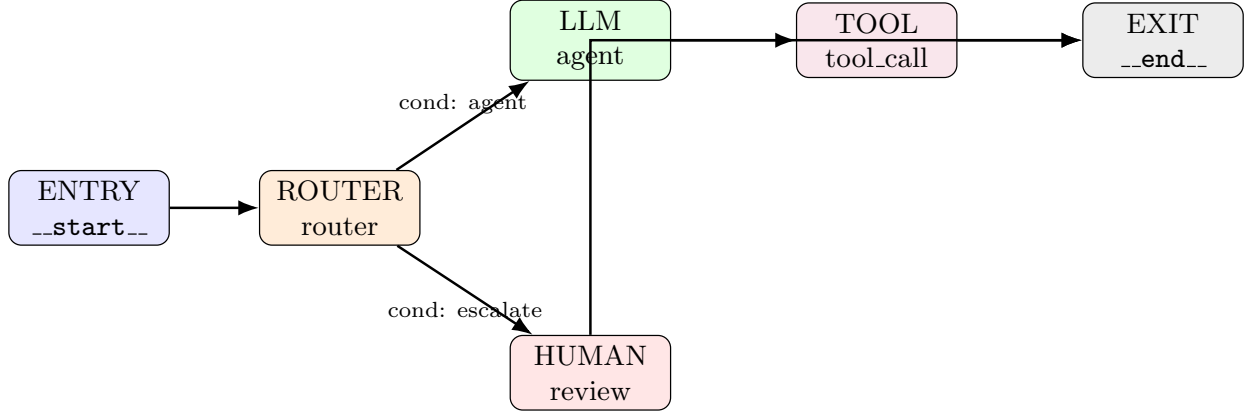


Figure 2: Example extracted workflow graph with typed nodes (ENTRY/ROUTER/LLM/TOOL/HUMAN/EXIT).

## 5 Verification methods

Agentproof provides two complementary verification layers: (i) structural checks over the extracted graph and (ii) temporal monitors evaluated over an event stream.

### 5.1 Structural checks

Structural checks operate over the adjacency structure of the extracted directed graph and the node/edge kind labels.

**Exit reachability.** We compute the set of nodes reachable from the entry using BFS/DFS and verify that all declared exit nodes are included. This is  $O(|V| + |E|)$ .

**Dead-end detection.** We identify nodes (excluding EXIT) with no outgoing edges. Dead ends often indicate missing transitions or incomplete error handling. This is  $O(|V| + |E|)$  with adjacency precomputation.

**Router shape checks.** Router nodes are expected to branch via conditional edges. We verify that all outgoing edges from a ROUTER are labeled CONDITIONAL. This is  $O(|E|)$ .

**Human-in-the-loop presence.** Some policies require at least one human approval node. We check whether the graph contains any node of kind HUMAN. This is  $O(|V|)$ .

**Tool declaration checks.** Tool nodes should declare the tool names they may call. We flag TOOL nodes with an empty tool list. This is  $O(|V|)$ .

## 5.2 Temporal policy monitors

Agentproof supports a small temporal policy DSL compiled into a DFA and evaluated with constant overhead per event.

**Event model.** Events are dictionaries with optional fields such as `tool_name`, `action_type`, `decision`, and a set of `tags`. Temporal predicates match against these fields (e.g., `tool:delete_account`).

**Supported DSL forms.** The implementation supports three forms:

- $G \neg \text{atom}$  (forbidden event): violation if `atom` matches any event.
- $a \rightarrow F b$  (interleaving constraint): once `a` occurs, `b` must occur before `a` occurs again.
- $a U b$  (until-like safety): `a` must hold on each step until `b` occurs; otherwise a violation is triggered.

These forms are intentionally restrictive; they capture common safety patterns while keeping compilation and evaluation simple.

**Compilation to DFA.** Each rule is compiled into a DFA with a small fixed number of states (three in the current implementation). DFA transitions are implemented as a precomputed table indexed by a bit-vector valuation of predicates. This yields  $O(1)$  state updates per event per rule.

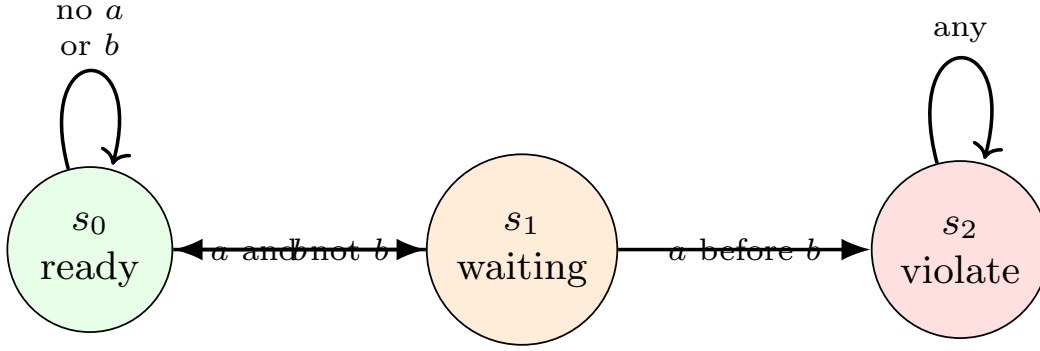
**Evaluation and handling.** At runtime, each event advances the DFA state for every compiled rule. If any rule enters a violation state, Agentproof maps the rule’s configured handling level (`warn`, `block`, `halt`, `escalate`) into an overall decision used by the caller.

## 6 Evaluation

We evaluate Agentproof along two axes: (i) graph extraction and structural statistics for representative agent workflows, and (ii) microbenchmarks for temporal monitor compilation and event evaluation throughput.

**Case studies.** We use three representative workflows built using real framework APIs: (i) a LangGraph incident response triage and mitigation graph, (ii) a hierarchical Google ADK compliance document pipeline (sequential + parallel + loop) with a human signoff step, and (iii) an AutoGen AgentChat change-control board topology with a human approver. These workflows are constructed without executing any external API calls.

**Methodology.** For each workflow, we extract an `AgentGraph` and compute node/edge counts as well as counts by node and edge kind. We measure extractor runtime as the median over 200 runs (after 20 warm-up runs) on the same in-memory workflow object. For temporal monitors, we measure compilation time as the median over 500 runs per DSL string and evaluation throughput (events/s) over a fixed synthetic 1,000-event stream. We also run Agentproof’s structural checks on each extracted graph and evaluate a small temporal policy suite over three representative traces per workflow (one safe, two unsafe).



Example DSL:  $a \rightarrow F b$   
(interleaving constraint)

Figure 3: Illustrative DFA for an  $a \rightarrow F b$ -style interleaving constraint: after  $a$  occurs,  $b$  must occur before  $a$  repeats.

Table 1: Graph statistics and extractor runtimes for representative workflows. Node-kind columns report counts of TOOL/LLM/ROUTER/HUMAN/SUBGRAPH nodes; other kinds (ENTRY/EXIT/PASSTHROUGH) are included in  $|V|$ .

| Study                    | $ V $ | $ E $ | Tool | LLM | Router | Human | Subg | Dir | Cond | Par | Loop | Med. ms |
|--------------------------|-------|-------|------|-----|--------|-------|------|-----|------|-----|------|---------|
| LangGraph (incident)     | 8     | 8     | 2    | 2   | 1      | 1     | 0    | 6   | 2    | 0   | 0    | 0.186   |
| ADK (compliance)         | 13    | 16    | 4    | 3   | 0      | 1     | 3    | 12  | 0    | 3   | 1    | 0.019   |
| AutoGen (change-control) | 6     | 5     | 0    | 3   | 0      | 1     | 0    | 5   | 0    | 0   | 0    | 0.007   |

**LangGraph incident response.** The incident response workflow uses a conditional router to branch into either automatic mitigation or a human approval step before executing mitigation tools. We verify  $G \text{ !tool:drop\_database and decision:mitigate } \rightarrow F \text{ action:human\_approval}$ . The happy-path trace passes, while unsafe traces halt on the forbidden tool and escalate on missing approval between repeated mitigation decisions (Table 2).

**ADK compliance pipeline.** The ADK pipeline composes sequential, parallel, and loop agents for document collection, analysis, remediation, and human signoff. We verify  $G \text{ !tool:exfiltrate\_data and read\_only U action:human\_signoff}$ . The monitors halt on exfiltration attempts and block writes prior to signoff.

**AutoGen change-control board.** The AutoGen case study models an acyclic speaker transition topology ending in a UserProxyAgent approver. We verify  $G \text{ !tool:send\_email and decision:deploy } \rightarrow F \text{ action:human\_approval}$ ; unsafe traces halt on email attempts and on repeated deploy decisions without a human approval event.

**Discussion.** Across case studies, extracted graphs are small enough that structural checks complete in time linear in graph size. Temporal monitoring is also lightweight: rules compile into constant-size

Table 2: Structural check results and temporal monitor outcomes over representative traces (one safe and two unsafe) for each case study.

| Study                    | Struct. (pass/total) | happy_path | forbidden_tool | policy_violation |
|--------------------------|----------------------|------------|----------------|------------------|
| LangGraph (incident)     | 5/5                  | PASS       | HALT           | ESCALATE         |
| ADK (compliance)         | 5/5                  | PASS       | HALT           | BLOCKED          |
| AutoGen (change-control) | 5/5                  | PASS       | HALT           | HALT             |

Table 3: Temporal monitor microbenchmarks. Compile time is the median over 500 runs per DSL string. Evaluation throughput is measured on a fixed 1,000-event synthetic stream (median over 50 runs).

| ID             | DSL / workload                                    | Preds | Compile ms (median) | Eval events/s (median) |
|----------------|---------------------------------------------------|-------|---------------------|------------------------|
| forbidden_tool | <code>G !tool:rm_rf</code>                        | 1     | 0.003               | –                      |
| interleave     | <code>action:fetch -&gt; F action:validate</code> | 2     | 0.005               | –                      |
| until_like     | <code>action:ingest U action:validate</code>      | 2     | 0.005               | –                      |
| evaluate       | 3 monitors on 1000 events                         | 3     | –                   | 327.94k                |

automata and evaluation performs a constant number of table lookups per event per rule.

## 7 Limitations and future work

**Workflow structure vs. LLM semantics.** Agentproof verifies properties of the workflow *topology* and the event stream emitted by execution. It does not prove semantic properties of LLM outputs (e.g., factuality or intent), which depend on non-deterministic model behavior and prompt/context choices.

**Extractor best-effort.** Frameworks evolve quickly and expose different internal representations across versions. The extractors in Agentproof are designed to be robust, but they inevitably involve heuristics (e.g., recognizing human review nodes by naming or metadata).

**Temporal DSL expressiveness.** The current temporal policy DSL is intentionally small and focused on common safety patterns; it is not a full LTL implementation. Extending to richer temporal fragments and to full graph  $\times$  automaton model checking is a natural next step.

**Library integration.** Some structural analyses are currently demonstrated in examples. Future work includes packaging a stable verification API that returns structured reports and counterexample traces, suitable for CI gating and developer feedback.

## 8 Conclusion

Agent frameworks already encode behavior as explicit workflow graphs. Agentproof leverages this structure to enable pre-deployment verification of safety properties without adding runtime overhead. By extracting a unified abstract graph model, running inexpensive structural checks, and



compiling temporal policies into deterministic monitors, Agentproof provides a practical safety layer for workflow-graph agents.

The repository implementation and examples demonstrate that meaningful safety properties can be checked without running LLM inference, making static verification a useful complement to runtime guardrails for production agents.

## References

- [1] AutoGen AgentChat. Python package `autogen-agentchat` (Microsoft). Accessed: 2026-02-28.
- [2] CrewAI documentation. <https://docs.crewai.com>. Accessed: 2026-02-28.
- [3] Google Agent Development Kit (ADK) documentation. <https://google.github.io/adk-docs/>. Accessed: 2026-02-28.
- [4] LangGraph documentation. <https://docs.langchain.com/oss/python/langgraph/overview>. Accessed: 2026-02-28.
- [5] Edmund M. Clarke, Orna Grumberg, and Doron A. Peled. *Model Checking*. MIT Press, 1999.
- [6] Amir Pnueli. The temporal logic of programs. In *18th Annual Symposium on Foundations of Computer Science (FOCS)*, 1977.
- [7] Moshe Y. Vardi. Reasoning about the past with two-way automata. *International Colloquium on Automata, Languages and Programming (ICALP)*, 1994.